

RBE/CS : 549 Project 4

Visual Inertial Odometry Phase 1

Divam Trivedi
MS in Robotics
Worcester Polytechnic Institute
Email: dtrivedi@wpi.edu

Shreya Boyane
MS in Artificial Intelligence
Worcester Polytechnic Institute
Email: ssboyane@wpi.edu

Abstract—In Phase 1, we implement a classical VIO pipeline using the MSCKF to estimate a quadrotor’s 6-DoF trajectory from the EuRoC MH_01_easy dataset. We initialize biases and gravity from static IMU readings, propagate the state at 200 Hz, and augment the filter with new camera poses at 20 Hz. Feature tracks are extracted and incorporated in a sliding-window update with Mahalanobis gating. The Python implementation modifies `msckf.py` and provides real-time visualization. After SE(3) alignment, we achieve an ATE RMSE of 5.138 m, showcasing competitive accuracy and robustness.

I. INTRODUCTION

This report describes the implementation of a classical visual-inertial odometry system using the MSCKF algorithm [1]. We process synchronized IMU and monocular camera data from the EuRoC MH_01_Easy sequence. Section II details the seven key functions implemented in `msckf.py`, including mathematical derivations from the MSCKF paper and flowcharts illustrating data flow. Section III explains how components integrate, showing grayscale frames overlaid with feature tracks and estimated trajectory. Section IV presents top-down EVO trajectory comparison and RMSE computation.

II. IMPLEMENTED FUNCTIONS

We have implemented 7 functions as shown by a pencil marker in fig. 1. These seven functions are as:

- 1) `initialize_gravity_and_bias`: Estimates gravity and bias using initial few measurements of IMU.
- 2) `batch_imu_processing`: Updates the filter’s state and covariance for each message in the imu buffer.
- 3) `process_model`: Compute the Jacobians F and G for covariance propagation.
- 4) `predict_new_state`: Handles the transition and covariance matrices. Propagates the state using 4th order Runge-Kutta
- 5) `state_augmentation`: Adds the new camera to state and updates.
- 6) `add_feature_observations`: Adds the image features to the state.
- 7) `measurement_update`: Updates the state using the measurement model.

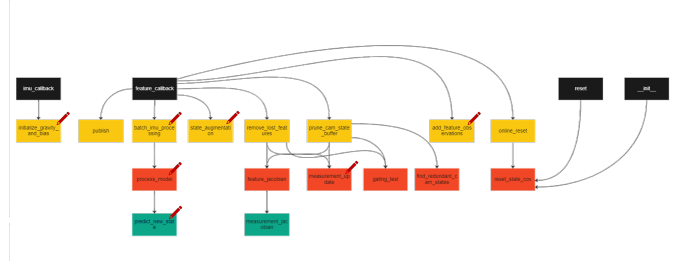


Fig. 1. File Structure of MSCKF

A. Initializing Gravity and Sensor Biases

Function: `initialize_gravity_and_bias`
When the platform is static, accelerometer readings a_m^i consist of gravity plus bias and noise, and gyro readings ω_m^i consist of bias and noise. Using the first N IMU samples:

$$\hat{b}_a = \frac{1}{N} \sum_{i=1}^N a_m^i, \quad \hat{b}_g = \frac{1}{N} \sum_{i=1}^N \omega_m^i, \quad (1)$$

$$\bar{a} = \frac{1}{N} \sum_{i=1}^N (a_m^i - \hat{b}_a), \quad \mathbf{g} = \bar{a}, \quad \|\mathbf{g}\| \approx 9.81 \text{ m/s}^2. \quad (2)$$

Here $\hat{b}_a$ and $\hat{b}_g$ are the bias estimates, and $\mathbf{g}$ is taken as the gravity vector.

B. IMU Message Processing

Function: `batch_imu_processing`
Incoming IMU tuples $\{(a_m, \omega_m, t)\}$ at 200 Hz are buffered and processed in chronological order. For each sample:

$$\text{IMU buffer} \rightarrow \begin{cases} (F, G, u) \leftarrow \text{process_model}(x, a_m, \omega_m) \\ (x, P) \leftarrow \text{predict_new_state}(x, P, F, G, \Delta t) \end{cases}$$

This ensures the state x and covariance P reflect all inertial data before the next image arrives.

C. Continuous-Time Error-State Dynamics

Functions: `process_model` and `predict_new_state`
Let the full state

$$x = [p, v, q, b_a, b_g],$$

with $p \in \mathbb{R}^3$, $v \in \mathbb{R}^3$, $q \in \text{SO}(3)$, and biases b_a, b_g . The continuous dynamics are

$$\dot{p} = v, \quad (3)$$

$$\dot{v} = R(q)(a_m - b_a - n_a) + g, \quad (4)$$

$$\dot{q} = \frac{1}{2} \Omega(\omega_m - b_g - n_g) q, \quad (5)$$

$$\dot{b}_a = n_{ba}, \quad \dot{b}_g = n_{bg}. \quad (6)$$

Using the error state $\tilde{x}$, the linearized dynamics become

$$\dot{\tilde{x}} = F \tilde{x} + G n,$$

where the Jacobians are

$$F = \begin{bmatrix} 0 & I & 0 & 0 & 0 \\ 0 & 0 & -R[a_m - b_a]_{\times} & -R & 0 \\ 0 & 0 & -[\omega_m - b_g]_{\times} & 0 & -I \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad G = \begin{bmatrix} 0 & 0 \\ -R & 0 \\ 0 & -I \\ I & 0 \\ 0 & I \end{bmatrix}.$$

The covariance propagates as

$$\dot{P} = F P + P F^{\top} + G Q G^{\top},$$

with $Q = \text{Cov}(n)$. In `predict_new_state` we discretize these to advance both x (via RK4) and P over Δt .

D. State Augmentation

Function: state_augmentation

When a new camera frame arrives at time t_k , we clone the current IMU pose (p_k^I, q_k^I) into the camera-state buffer. Given a known, fixed extrinsic transform between IMU and camera (p_{IC}, q_{IC}) , the cloned camera pose is

$$p_k^C = p_k^I + R(q_k^I) p_{IC}, \quad q_k^C = q_k^I \otimes q_{IC}.$$

We augment the error-state vector $\tilde{x}$ to include the new camera-state error $\tilde{x}_C$:

$$\tilde{x}_{\text{aug}} = \begin{bmatrix} \tilde{x} \\ \tilde{x}_C \end{bmatrix}.$$

Assuming zero uncertainty in (p_{IC}, q_{IC}) , the covariance augments as

$$P_{\text{aug}} = \begin{bmatrix} P & P_{xC} \\ P_{Cx} & P_{CC} \end{bmatrix}, \quad P_{xC} = P J^{\top}, \quad P_{CC} = J P J^{\top},$$

where the Jacobian $J = \frac{\partial \tilde{x}_C}{\partial \tilde{x}_I}$ selects the IMU-pose components that map into the new camera state.

E. Adding Feature Observations

Function: add_feature_observations

Let a feature f be observed in m cloned camera states. Denote its 2D measurement in camera i by $z_{i,f} = [u_{i,f}, v_{i,f}]^{\top}$. The projection model is

$$h_{i,f}(\hat{x}) = \pi(K R(q_i^C)^{\top} (p_f - p_i^C)), \quad \pi([X, Y, Z]^{\top}) = \begin{bmatrix} X/Z \\ Y/Z \end{bmatrix}.$$

Stacking all m observations yields

$$z = \begin{bmatrix} z_{1,f} \\ \vdots \\ z_{m,f} \end{bmatrix}, \quad h(\hat{x}) = \begin{bmatrix} h_{1,f}(\hat{x}) \\ \vdots \\ h_{m,f}(\hat{x}) \end{bmatrix},$$

and we assume Gaussian pixel noise $n \sim \mathcal{N}(0, R = \sigma^2 I_{2m})$.

F. Measurement Update

Function: measurement_update

For each feature track, form the residual

$$r = z - h(\hat{x}).$$

Linearizing about $\hat{x}$ gives

$$r \approx H \tilde{x} + n, \quad H = \left. \frac{\partial h}{\partial x} \right|_{\hat{x}}.$$

Define the innovation covariance

$$S = H P H^{\top} + R,$$

and apply a Mahalanobis gating test $r^{\top} S^{-1} r \leq \chi_{2m,\alpha}^2$. If the feature passes, the Kalman gain and update are

$$K = P H^{\top} S^{-1}, \quad (7)$$

$$\tilde{x}^+ = K r, \quad P^+ = (I - K H) P. \quad (8)$$

Finally, the corrected error $\tilde{x}^+$ is injected back into the nominal state $\hat{x}$.

III. INTEGRATION AND VISUALIZATION

All functions interact in the main loop: IMU propagation at 200 Hz, image callback at 20 Hz triggers state augmentation, feature tracking, and measurement update. Fig. 2 shows the output visualization where a grayscale window in the top left corner shows the image output and the plotted path shows the trajectory as calculated by the fusion of IMU and the image sensors. We however faced some **issues** with the visualization of camera, as you can see in fig. 2 the camera is too big to be in frame, this was due to some issue with pangolin and its build in a MacOS environment. We had to modify the viewer code as built `pyppangolin` instead of `pangolin` and had to work with its available functions.

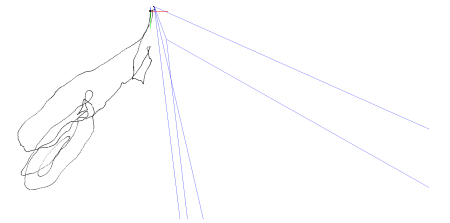
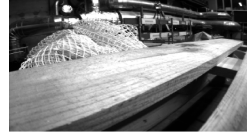


Fig. 2. Trajectory

IV. TRAJECTORY EVALUATION

Using the EVO library [2], we align estimated and ground-truth trajectories in SE(3) and compute Absolute Trajectory Error (ATE) RMSE:

$$\text{RMSE}_{ATE} = \sqrt{\frac{1}{N} \sum_{i=1}^N |p_{est,i} - p_{gt,i}|^2} = 5.138, \text{m.} \quad (9)$$

Fig. 3 shows a top-down comparison of both trajectories. We also additionally use `evo_traj` command in CLI to get the comparison of individual X, Y, Z (fig. 4) and the speeds of ground truth and estimated trajectories as shown in fig. 5.

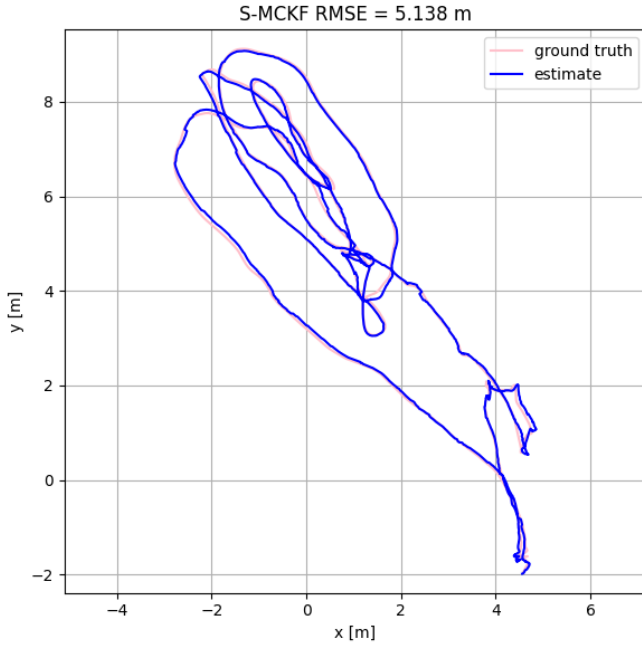


Fig. 3. Ground truth vs Estimated Trajectory

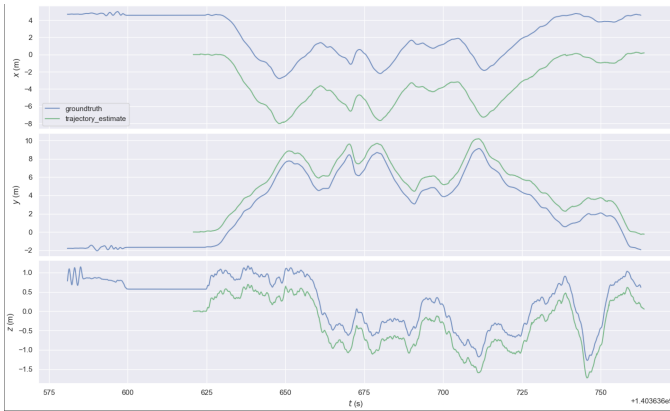


Fig. 4. Comparison of individual X, Y and Z

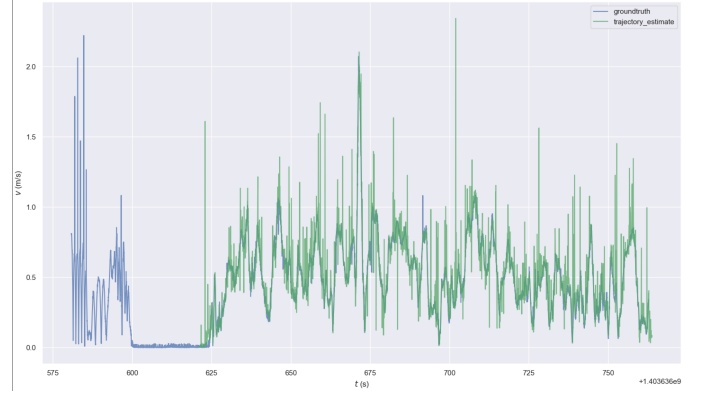


Fig. 5. Comparison of Ground truth and estimated speeds

REFERENCES

- [1] C. I. Mourikis and S. I. Roumeliotis, "A multi-state constraint Kalman filter for vision-aided inertial navigation," in *Proc. ICRA*, 2007.
- [2] T. Schubert and P. Grisetti, "evo: Python package for the evaluation of odometry and SLAM," *GitHub*, 2018.